Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION TECHNOLOGY



Project Title: Klaro: A Private, Offline, Multilingual Assistant for Reading Tender Documents (RFPs)

Enrol. No.	**Name of Student**
9923103059	Yasharth Singh
9923103069	Nikunj Sethi
9923103072	Shrey Sircar

Course Name: Major Project Part-1

Course Code: 15B19CI791

Program: B. Tech. CSE

4th Year 7th Sem

2026 - 2027

ACKNOWLEDGEMENT

We would like to place on record our deep sense of gratitude to Prof. Shikha Mehta, HOD-CSE, Jaypee Institute of Information Technology, India, for her generous guidance, help and useful suggestions given to us throughout this project.

We express our sincere gratitude to her for the stimulating guidance, continuous encouragement and supervision throughout the course of the present work.

We also wish to extend our thanks to our classmates for their insightful comments and constructive suggestions to improve the quality of this project work.

Signature(s) of Students

Yasharth Singh	Nikunj Sethi	Shrey Sircar
(9923103059)	(9923103069)	(9923103072)

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and beliefs, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma from a university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place: Noida
Date: _____

Name: Yasharth Singh
Enrolment No.: 9923103059

Name: Nikunj Sethi
Enrolment No.: 9923103069

Name: Shrey Sircar
Enrolment No.: 9923103072

CERTIFICATE

This is to certify that the work titled "Klaro: A Private, Offline, Multilingual Assistant for Reading Tender Documents (RFPs)" submitted by Yasharth Singh, Nikunj Sethi and Shrey Sircar of B.Tech of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of any other degree or diploma.

_____

Signature of Supervisor

Name of Supervisor: Prof. Shikha Mehta
Designation: HOD-CSE
Date: _____

ABSTRACT

When a company wants to bid for a government or corporate contract, it first has to read the tender document, called a Request for Proposal (RFP). These documents often run to 50 to 100 pages, and the details that matter (who is asking, what work is wanted, who is allowed to bid, deadlines, payment terms, penalties) are scattered across them. RFPs are also confidential, so uploading them to an online AI service is often not allowed. Many tenders are written in languages other than English. We have chosen to include German translation in Klaro as of now.

Klaro is a desktop application that reads an RFP and writes a short summary under nine headings entirely on the user's own laptop. Nothing is sent over the internet. It uses a small AI language model (Qwen3.5 with 4 billion parameters) that runs locally. A German RFP is first translated to English on the laptop, and the finished summary can be shown in German. Every point shows the page it came from. Clicking it opens the original PDF and highlights the passage it most likely came from. The dates in a summary can be saved as a calendar file.

We compared three small models and chose Qwen3.5. Full runs on English RFPs of 54 and 84 pages took 22 to 25 minutes, and runs on German RFPs of 24 and 31 pages took 9 to 11 minutes. We also found a fault: with the default memory setting, the summary step saw only about 12% of the facts (2,050 of 17,496 tokens in one test), and the code filled the gaps by copying facts. Raising the setting cut the average summary of 11 documents from 4,565 to 545 words, but made each run about three times slower. On one German run it also cut the share of facts used from 98% to 65%. We built a set of 32 draft summaries (written by an AI and checked by machine, not yet by a human) and a LoRA fine-tuning kit. The kit has not been trained on the real 4-billion-parameter model.

Keywords: tender documents, RFP, offline AI, small language model, document summary, machine translation, German, privacy.

Table of Contents

Page No.

Acknowledgement . . . . . *i*

Declaration . . . . . *ii*

Certificate . . . . . *iii*

Abstract . . . . . *iv*

List of Tables . . . . . *vii*

List of Figures . . . . . *viii*

List of Abbreviations . . . . . *ix*

List of Nomenclature . . . . . *x*

Chapter 1: INTRODUCTION

1.1 General 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope of the Present Work 1

Chapter 2: BACKGROUND STUDY

2.1 General 2

2.2 Earlier Work on Tender Documents 2

2.3 Reading Very Long Documents 2

2.4 Small AI Models and Made-Up Answers 2

2.5 Machine Translation 2

Chapter 3: REQUIREMENT ANALYSIS

3.1 Who Uses Klaro 3

3.2 What the System Must Do 3

3.3 Quality Requirements 3

3.4 Hardware and Software Requirements 3

Chapter 4: DETAILED DESIGN

4.1 How the Parts Fit Together 4

4.2 The Document Processing Pipeline 4

4.3 The German Path 4

4.4 What the Summary Contains 5

4.5 Keeping the Document Safe 5

Chapter 5: IMPLEMENTATION

5.1 The Seven Steps in Short 6

5.2 German Detection and Translation 6

5.3 Source Highlighting 6

5.4 English to German View 6

5.5 Calendar Export 7

5.6 The Desktop Application 7

5.7 Golden Dataset 7
5.8 LoRA Fine-Tuning Kit 8
5.9 The Context-Size Fix 8
5.10 One-Command Setup 8
5.11 Testing 8

Chapter 6: EXPERIMENTAL RESULTS AND ANALYSIS

6.1 Choosing the AI Model 9
6.2 Full Runs through the Desktop App 9
6.3 The Context-Size Experiment 10
6.4 German Results 10

Chapter 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion 14
7.2 Limitations 14
7.3 Future Scope 14

References **15**

List of Tables

Table	Title	Page
3.1	Functional requirements	3
3.2	Hardware and software requirements	3
4.1	The nine headings of the summary	5
6.1	Model benchmark on 20 RFP parts with 47 hand-marked requirements	9
6.2	Results of the full runs (Apple M4 laptop)	9
6.3	Effect of the context size on the summary (11 documents, average per document)	10

List of Figures

Figure	Title	Page
1.1	Klaro in one picture	1
4.1	The document processing pipeline	4
4.2	How a German RFP moves through Klaro	4
5.1	Home screen with past analyses and model status (top); progress screen during a German run (bottom)	7
6.1	English RFP: the summary beside the original PDF. Clicking a page number opens that page (screenshot from an earlier run, before passage highlighting)	11
6.2	Results screen with the automatic quality checks and a flagged bullet	11
6.3	German RFP in the English view, with the "Translated from German" banner and a clicked point whose passage is highlighted in the PDF (top); the German view of another run (bottom)	12
6.4	The "Add to calendar" preview, with items that need review	13

List of Abbreviations

4B	4 billion parameters (a measure of an AI model's size)
AI	Artificial Intelligence
GB, GiB	Gigabyte; gibibyte (about 1.07 GB). Both measure memory
ICS	iCalendar file format (`.ics`) used by calendar programs
LoRA	Low-Rank Adaptation (a low-cost way to further train a model)
OPUS-MT	Open translation models from the Helsinki-NLP group
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RFP	Request for Proposal (a tender document)
TF-IDF	Term Frequency–Inverse Document Frequency (a keyword-scoring method)

List of Nomenclature

Agent	An AI helper with one job, for example writing the summary
Part (chunk)	A piece of the RFP of about 1,800 words that the AI reads at one time
Fact	One short statement taken from the RFP, with its page number
Executive summary	The short overview of the RFP under nine fixed headings
Pipeline	The fixed sequence of steps every RFP goes through
Local model	An AI model that runs on the user's own computer, not on the internet
Word piece (token)	The small unit of text an AI model reads and writes, often part of a word
Context size	How many word pieces the model can look at in one call (Ollama default here: 4,096)
Segment	One sentence or heading of the PDF text, translated on its own
Flag	A mark that something may be wrong (for example a number missing after translation). Never fixed automatically
Golden summary	A reference summary written to a fixed standard, used for training and testing
Adapter	The small set of extra numbers that LoRA training produces; it is added to the base model
Made-up information	A statement from the AI that is not in the document (also called hallucination)
Precision, recall	Of the items the AI listed, the share that were correct; of the items that should have been listed, the share it found
Benchmark	A fair test that gives several models the same task and compares their scores

Chapter 1: INTRODUCTION

1.1 General

A Request for Proposal (RFP) is a document an organization publishes when it wants companies to bid for a piece of work. Before a bid team can decide whether to respond, someone has to read the whole RFP and note who issued it, what work is required, who can bid, the deadlines, payment terms, contract length, and special conditions such as penalties. These points are spread across many pages.

AI chatbots can summarize long documents, but that means uploading the document to someone else's servers. Bid teams usually cannot do that with confidential tenders. Klaro takes a different approach: the AI model runs on the user's own laptop, so the document never leaves it (Figure 1.1). Many tenders in Europe are in German, so Klaro also reads German RFPs.

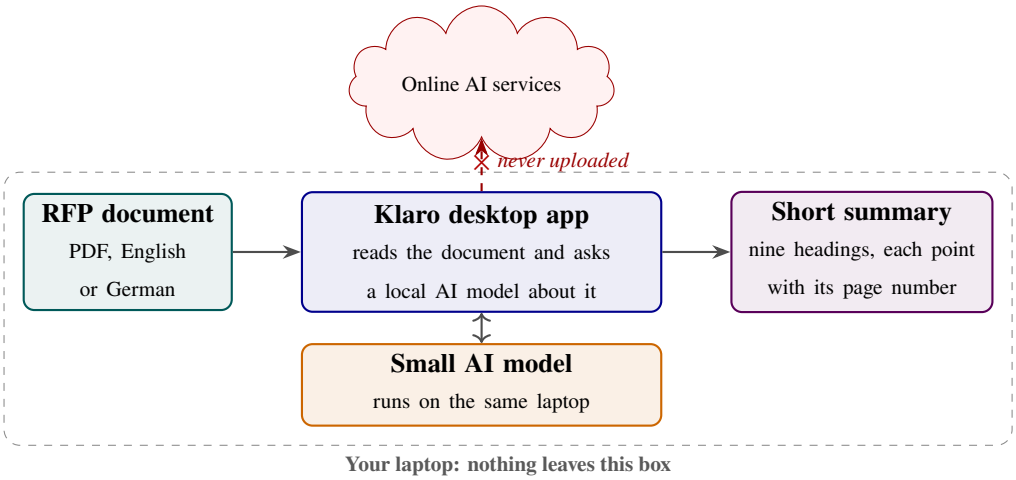


Figure 1.1: Klaro in one picture

1.2 Problem Statement

Given a tender document (RFP) as a PDF in English or German, produce a short summary under nine fixed headings, where every point shows which page it came from. It must run on an ordinary laptop, use only an AI model that runs on that laptop, and never send any part of the document over the internet. A non-technical reader must be able to check each point against the original, and when something goes wrong the app must say so clearly instead of showing a half-finished result.

1.3 Objectives

1. Build a step-by-step process that turns an RFP PDF into a page-referenced summary.
2. Pick the most suitable small AI model by testing several on the same task.
3. Measure the quality of the summaries and record the weak points honestly.
4. Package everything as a desktop app with progress updates and a side-by-side view of the original.
5. Accept German RFPs, translate them on the laptop, and show where each point came from in the PDF.
6. Prepare a reference dataset and a fine-tuning kit for improving the summary step.

1.4 Scope of the Present Work

Klaro handles English and German RFPs in text-based PDF form and one feature, the executive summary. Scanned files, Word files, other languages, requirement checklists, proposal drafts and search over past proposals are left for later (Chapter 7).

Chapter 2: BACKGROUND STUDY

2.1 General

Computers have been used to analyze tender documents since before today's AI chatbots. Older methods counted important words and picked out the most important sentences. Newer methods use AI models that can read long text. We studied papers close to our problem, plus background work on the kind of AI model and translation system we use.

2.2 Earlier Work on Tender Documents

Beason et al. [1] processed a large set of U.S. Department of Defense and Homeland Security RFPs to spot technology trends. They found that RFPs follow no common layout and contain a lot of noise (contents pages, boilerplate, legal text), which has to be cleaned out first. They also found that exact, structured facts are more useful to readers than a free-flowing summary. Klaro follows both points: it skips contents pages and produces fixed headings.

Trappey et al. [2] summarized more than 1,300 tenders for power transformers. They found key terms by counting words (TF-IDF), grouped sentences by topic, and picked the highest-ranked sentences with TextRank. Keeping the top 35% of sentences preserved about 80% of the important requirements and cut reading time to about 29%. We took two ideas from this: summarize topic by topic rather than all at once, and aim for a real reduction in length. Knisely and Pavliscsak [3] sorted research proposals into themes. A small number of proposals were labelled by experts, and those labels guided the sorting of the rest, which worked much better than sorting with no human input. This is why we built a set of reference summaries (Chapter 5).

2.3 Reading Very Long Documents

Standard AI models can only look at a limited amount of text at once, and the cost grows quickly as the text gets longer [4]. Longformer [5] reduces this cost so a model can read thousands of words in one go. Douzon et al. [6] also used the page layout of business documents, so that a table value stays linked to its heading. A small model on a laptop cannot read an 84-page RFP in one go, so Klaro cuts it into pieces, keeps page numbers on every piece and combines the results afterwards.

2.4 Small AI Models and Made-Up Answers

Language models sometimes state things that are not in the text they were given [7, 8], and they pay less attention to the middle of very long inputs [9]. One remedy is to first look up the relevant passages and give only those to the model (retrieval-augmented generation, [10]). Klaro's remedy is to make the model cite a page for every point and then check each point against the collected facts. A model can be specialized for a task with light-weight fine-tuning such as LoRA [11], which trains a small add-on instead of the whole model.

2.5 Machine Translation

Machine translation turns text from one language into another with a trained model. OPUS-MT [12] is a set of small, open translation models, one per language direction. They are small enough to run on a laptop without internet, which is why we use them. We found no earlier work that combines offline translation, a small summary model and page-level checking for tenders on a laptop. The papers above handle English documents only, or need powerful hardware.

Chapter 3: REQUIREMENT ANALYSIS

3.1 Who Uses Klaro

The main user is a bid or proposal analyst who receives an RFP, in English or German, and has to brief a manager on it. This person is not expected to use programming tools. The project team also runs the same process from the command line to test changes.

3.2 What the System Must Do

Table 3.1: Functional requirements

No.	The system must…
R1	accept a text-based PDF in English or German and politely reject anything else;
R2	read the text page by page, remember the page numbers, and split it into parts for the AI model;
R3	translate German pages to English on the laptop before analysis, and flag numbers that did not survive;
R4	pull out facts from each part with the local AI model, combine them and remove repeats;
R5	write a summary under nine headings, with page numbers for every point;
R6	check the quality of the summary automatically;
R7	show progress while it works, and allow the user to cancel;
R8	show the summary beside the original PDF, and highlight the passage a clicked point came from;
R9	show the summary in German on request;
R10	export the dates in the summary as a calendar file;
R11	keep a history of past analyses, and check at start-up that the AI model is installed.

3.3 Quality Requirements

- **Privacy:** no internet connection is used while analysing; fonts and models are on the laptop.
- **Works on Windows and macOS:** the same app runs on both.
- **Honest about errors:** if a step fails, the app says which step and why.
- **Repeatable:** every analysis is saved in its own folder with the model used and the time of each step.
- **Small training machine:** the fine-tuning kit must fit a graphics card with 8 GB memory, which is what the team has.

3.4 Hardware and Software Requirements

Table 3.2: Hardware and software requirements

Item	Requirement (what we tested on)
Computer	At least 8 GB memory; tested on an Apple M4 laptop with 24 GB
Operating system	Windows 10/11 or macOS; tested on macOS 26.5 only
AI model runner	Ollama [13] with the `qwen3.5:4b` model [14] (about 3.4 GB)
Translation	OPUS-MT German-English model, about 301 MB, downloaded once
Analysis engine	Python 3.11 or newer, with pypdf [15], Pydantic [16] and FastAPI [17]
Desktop app	Electron [18] with React

Chapter 4: DETAILED DESIGN

4.1 How the Parts Fit Together

The user works in a desktop app. When the app opens, it quietly starts a local server. When the user adds an RFP, the server starts the analysis as a separate background job, so the app stays responsive and the job can be cancelled at any time. Only one analysis runs at a time, because the laptop runs a single AI model; others wait in a queue. The job saves everything it produces in its own folder. We did not use a workflow engine such as LangGraph [19]: the steps are a fixed sequence, and a few lines of Python run them.

4.2 The Document Processing Pipeline

Every analysis goes through the seven steps in Figure 4.1. Two of them use the AI model; the others are ordinary program logic, which is fast and always gives the same answer. A long RFP is cut into parts of about 1,800 words (roughly four pages) that overlap by 200 words, so a sentence cut at a boundary is not lost. Each part is read separately, its facts are noted with page numbers, and the notes from all parts are combined into one list for the summary step.

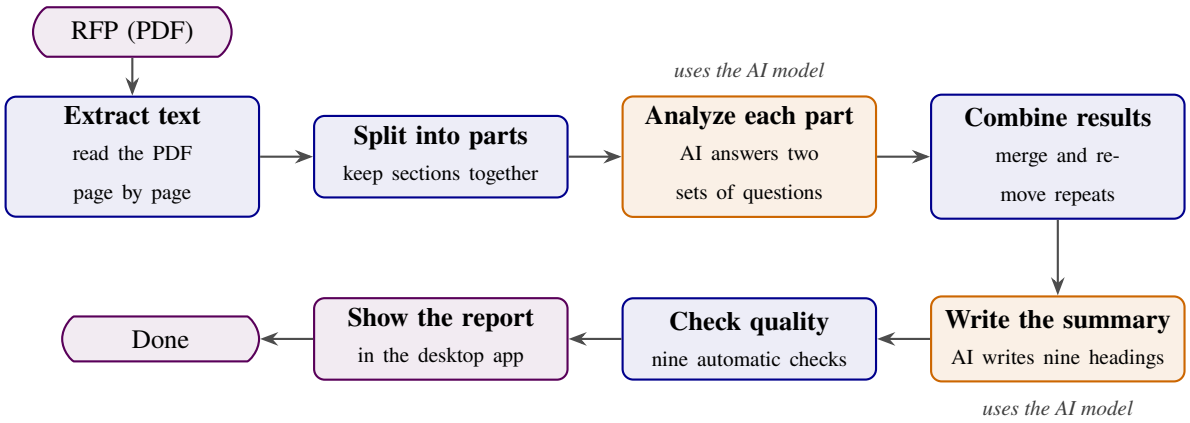


Figure 4.1: The document processing pipeline

4.3 The German Path

For a German RFP, an extra step is added after step 1 (Figure 4.2). The language is detected first. German pages are translated to English on the laptop, and then the unchanged English pipeline runs on the translated text. German text is never given to the English prompts. After the summary is written, each point is linked to a passage on its cited page. The German view of the summary is made only when the user asks for it.

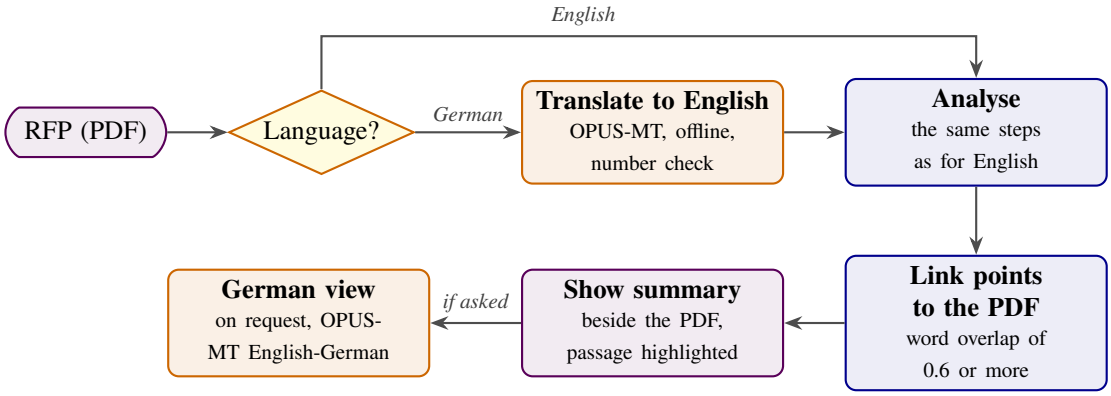


Figure 4.2: How a German RFP moves through Klaro

4.4 What the Summary Contains

Table 4.1: The nine headings of the summary

Heading	What goes under it
Overview	What is being bought and why, in two or three sentences
Issuing organization	Who published the RFP
Scope	The work and deliverables required
Key requirements	Documents and forms the bidder must submit
Eligibility	Who may bid: certificates, turnover, experience
Important dates	Deadlines, how long a bid stays valid, review dates
Commercial requirements	Pricing, billing, payment terms, taxes, deposits
Project duration	How long the contract runs, and extensions
Important conditions	Penalties, confidentiality, ownership of work, exit and termination rules

4.5 Keeping the Document Safe

The app only connects to programs on the same laptop: its own local server and the AI model runner. The screen part of the app cannot read or write files by itself; it can only ask the main part of the app for a small set of things, such as the server address, a file-picker dialog or a saved calendar file. The PDF viewer is allowed to show only the RFP of the current analysis. The summary panel can only ask the app to jump to a page number that exists in the document. One button, “Google Calendar”, opens a web page in the browser, and only when clicked; it sends that one event’s title and date to Google.

Chapter 5: IMPLEMENTATION

5.1 The Seven Steps in Short

Text is read from each page with pypdf and stored with its page number; contents pages are skipped. The first attempt asked the model for everything in a part with one question. On five parts, two answers were empty and one was cut off, so each part is now read twice: once for overview, scope, requirements and eligibility, and once for dates, money, duration and legal conditions. Answers must be in a fixed format with a page number for every point, and Pydantic checks the format. A slightly malformed answer is repaired instead of thrown away. If the model fails on more than half of the parts, the analysis is marked as failed. The facts are then grouped by heading and repeats are removed. The issuing organization is the name found most often (names ending in Bank, Authority or Ministry), and the answer is "Not identified" when Klaro is unsure. The summary step writes the nine headings from the combined facts, and an empty heading is filled by copying its facts. Nine automatic checks follow (structure, support by facts, page numbers, coverage, repeats, length, wrong heading and others). They compare words, so they show whether the summary agrees with the extracted facts, not whether the extraction was right.

5.2 German Detection and Translation

The language is guessed by counting common German and English words. If the text is too short or in another language, the run stops and asks the user to choose the language. German text is cut into segments (sentences and headings), and each segment is translated with the OPUS-MT German-English model [12]. The model runs offline; it is downloaded once during setup. Translated segments are cached by a hash of their text, so a repeat run is faster. After translation, every number in a German segment is looked for in its English segment, and a missing number sets a flag. Flags are shown, never fixed by the program. If the model or a library is missing, the run fails at once with a message that names the command that fixes it. Page numbers always stay the physical pages of the original PDF.

5.3 Source Highlighting

The facts carry only a page number, and the summary step rewrites them, so there is no stored link from a summary point to its source sentence. Klaro therefore builds the link after the summary is written. It scores windows of one to three neighbouring segments on the cited page by word overlap with the point (the first five letters of each word are compared) and keeps the best window if the score is 0.6 or more. The quote, its German original and the score go into `evidence.json`. Below 0.6 no quote is claimed, and the app says the exact passage was not found on that page. The viewer draws the PDF with pdf.js and highlights whole text lines, not exact characters.

5.4 English to German View

On request, the finished summary is translated sentence by sentence with the OPUS-MT English-German model. Only the text changes; headings, order, page numbers and the issuer name stay as they are. Numbers, dates and currency signs are compared with the English text, and differences are flagged in a separate file. The result is written to a staging folder and moved into place at the end, so a cancelled translation leaves nothing behind.

5.5 Calendar Export

The dates are read from the “Important dates” bullets, and from other bullets only when they contain an event word such as deadline, submission or opening (in English or German). Month names in both languages, ISO dates, ranges and 12-hour or 24-hour times are understood. A numeric date such as 03/04/2026 is read day first and marked “assumed” when both numbers are 12 or less and differ. A date with no year, no day or an impossible value goes to a “needs review” list. Klaro never guesses a year. The user sees a preview and saves an `.ics` file, which any calendar program opens.

5.6 The Desktop Application

The app starts the local server, waits until it is ready and shows a setup screen with the fix if the AI model is missing. Figure 5.1 shows the home and progress screens, and Figures 6.1 to 6.4 show the results screens.

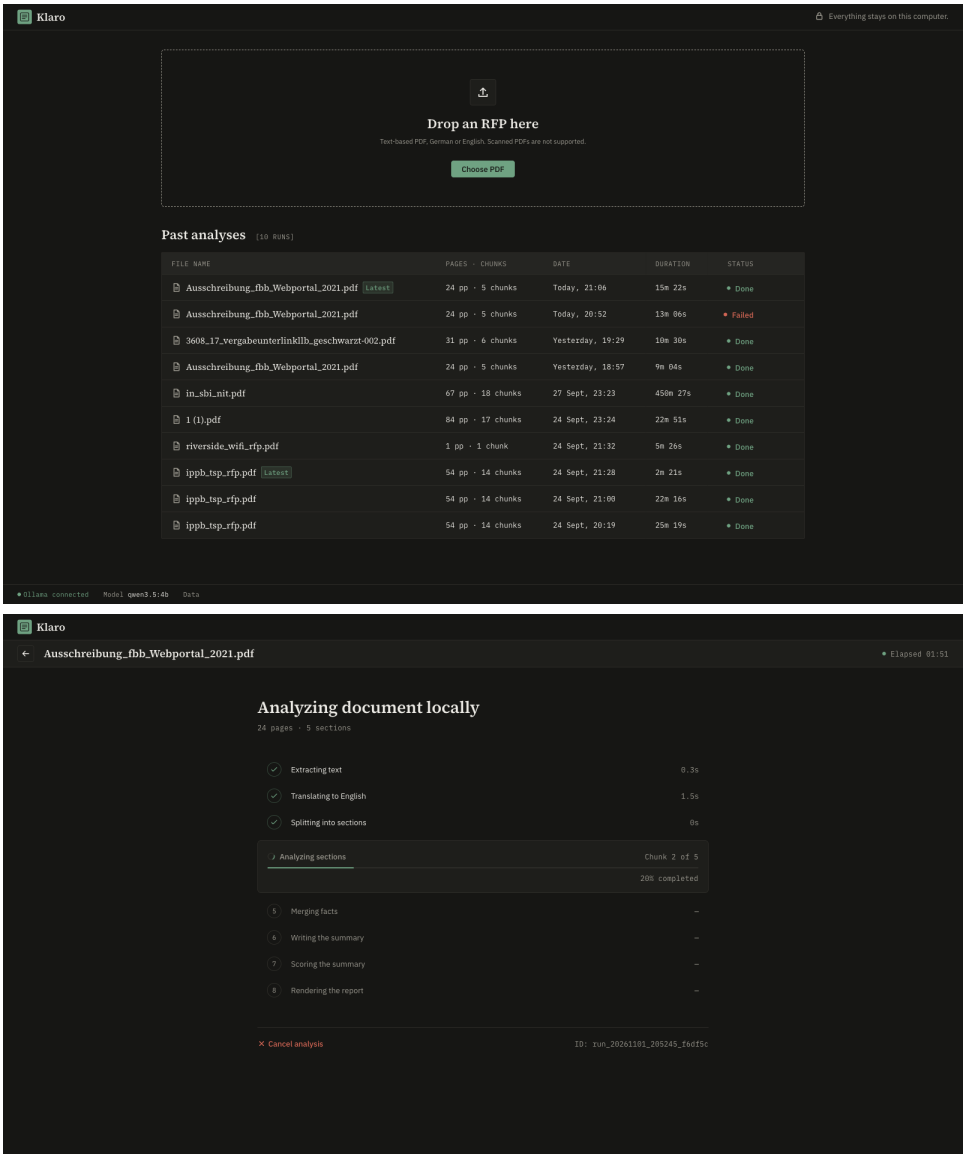


Figure 5.1: Home screen with past analyses and model status (top); progress screen during a German run (bottom)

5.7 Golden Dataset

We collected 45 English RFPs from public sources in 8 sectors (2,583 pages in total, 21 to 111 pages each, median 57). Of these, 35 are for training and 10 for testing, split by sector with a fixed random seed. A written standard (nine sections, a page for every bullet, numbers copied exactly, “Not specified in the document” when absent) tells how each golden summary is written. 32 documents have a summary so

far: 22 for training and all 10 for testing. The summaries were drafted by an AI model from the extracted text and passed automatic checks. They are therefore called *auto-checked*. **None has been checked by a human yet**, so they must not be called verified. A guide for the human check (20 to 25 minutes per document) is in the repository.

5.8 LoRA Fine-Tuning Kit

The kit trains a LoRA adapter [11] on the golden summaries (training split only), merges it into the model, loads it into Ollama and scores it on the 10 test documents. Training reads the training split only, and the scoring script refuses a document that was used for training. Adapters are saved as safetensors only, and checkpoint files that use Python's pickle format are deleted after a run. The kit was run only on a Mac with the 0.8-billion-parameter model of the same family: the training loss fell from 1.62 to 0.20 over five epochs and the merged model loaded in Ollama. **Nothing was trained or tested on the real 4-billion-parameter model.**

The 8 GB limit. Our graphics card has 8 GB. A whole-summary example is too long for it: the median document is 34,597 tokens, and only 1 of 32 is under 14,000. We estimate (plus or minus 30%, no GPU was used) that 4-bit training of the 4B model needs 14.7 GiB at 32,000 tokens and 7.8 GiB even at 6,000. So we changed the design to *per-section* examples: the pages one section cites, plus that section's rules, as input, and that section's bullets as output, at most 4,096 tokens. This gives 322 training, 39 validation and 165 test examples, and we estimate 7.3 GiB for the 4B model (borderline on 8 GB) and 5.2 GiB for a 2B fallback. The catch is that the app writes the whole summary in one call, so this adapter cannot be used in the app yet (Chapter 7).

5.9 The Context-Size Fix

Ollama's default context here is 4,096 word pieces. The summary step sends all the combined facts in one prompt. On the first test document the facts were 17,496 tokens, but the model saw 2,050: the rest was cut off silently. The setting `KLARO_NUM_CTX` now lets the summary step ask for a larger context. Its default is 0, which keeps the old behaviour, so the app only gets the fix when the variable is set (we set it to 32768 for the new German run).

5.10 One-Command Setup

After `npm install`, the command `npm run dev` first runs a setup script. It finds Python 3.11 or newer, creates a virtual environment, installs the pinned packages when the list changed, downloads the two translation models (a failed English-German download only warns), and checks Ollama. On a clean copy on a Mac the setup took 93 s, of which 74 s was the model download. It has not been run on Windows.

5.11 Testing

68 JavaScript tests (54 of them for the calendar) and 43 Python tests pass. The checks include the report pages, step order, failed parts, issuer detection, translation, highlighting and dates. The app was driven by hand on macOS; the screenshots in this report were taken from the running app. Automated tests do not drive the real window. A teammate is to test on Windows.

Chapter 6: EXPERIMENTAL RESULTS AND ANALYSIS

6.1 Choosing the AI Model

We compared three small models that can run on a laptop: Gemma 4 (e4b), Qwen3.5 (4B) and Phi-4-mini. Each was given the same 20 RFP parts and asked to list the requirements in them. Team members had marked 47 requirements in those parts by hand. A requirement counted as found when the model's wording matched the hand-marked one at 80% similarity. Table 6.1 shows the results.

Table 6.1: Model benchmark on 20 RFP parts with 47 hand-marked requirements

	Gemma 4 (e4b)	**Qwen3.5 (4B)**	Phi-4-mini
Requirements found (out of 47)	33	23	7
Wrong or extra items listed	22	**8**	9
Precision / recall	60% / 70%	**74%** / 49%	44% / 15%
F1-score	**65%**	59%	22%
Mandatory vs. optional judged correctly	85%	**91%**	57%
Made-up requirements	0%	0%	6%
Download size (GB) / parameters (billions)	9.6 / 8.0	3.4 / 4.7	2.5 / 3.8
Typical time per part (s) / peak memory (GB)	6.7 / 4.4	8.1 / 6.0	1.3 / 4.3
Answer in exactly the requested format	45%	0%	35%

Phi-4-mini was fastest but found only 7 of 47 requirements and was the only model that made one up. Gemma found more requirements and had the higher F1-score, but it is much bigger: a 9.6 GB download against Qwen's 3.4 GB. Qwen was the more careful model: right 74% of the time when it listed something against Gemma's 60%, with 8 wrong items against 22. A wrong point in a summary does more harm than a missing one, because a missing point can be found by asking more questions (Klaro asks two sets per part). Qwen's weak format-following is handled by Klaro's tolerant reading of answers.

6.2 Full Runs through the Desktop App

We ran the finished app on three English runs (R1 to R3: an 84-page government RFP, and a 54-page bank RFP run twice) and on two German RFPs through the normal app path (G1 and G2). A third German run (G3) repeats G1 with `KLARO_NUM_CTX=32768`. Its first attempt failed: the summary step ran past the default limit of 300 s (`KLARO_LLM_TIMEOUT`) and the run ended after 787 s with "timed out". The retry with the limit set to 900 s finished. Table 6.2 shows the results. R1 to R3 and G1, G2 used the default context.

Table 6.2: Results of the full runs (Apple M4 laptop)

	R1	R2	R3	G1	G2	G3
Language	EN	EN	EN	DE	DE	DE
Pages / parts	84 / 17	54 / 14	54 / 14	24 / 5	31 / 6	24 / 5
Total time	22m51s	25m19s	22m16s	9m04s	10m30s	15m21s
Translation time (s)	–	–	–	43.4	0.8	1.4
Points in the summary	170	211	216	104	137	40
Backed by facts / with page numbers	100/100%	100/100%	100/100%	100/100%	100/100%	97.5/100%
Facts included in the summary	99%	97%	96%	98.3%	96.8%	64.8%
Shorter than the facts by	8%	8%	9%	9.3%	9.9%	62.4%
Made-up information / failed parts	none / 0	none / 0	none / 0	none / 0	none / 0	1 flagged / 0

6.2.1 Time Taken

About 75% to 95% of the time goes into step 3, where the AI reads each part twice (G1: 446 s of 544 s; G2: 564 s of 630 s; G3: 687 s of 921 s). Writing the summary takes one more minute (54 s in G1, 65 s in G2). An English 54 to 84 page RFP takes 22 to 25 minutes, and the shorter German RFPs 9 to 11 minutes. The progress screen shows which part is being read.

6.2.2 Same Document, Different Answers

R2 and R3 were the same PDF, but the AI found 236 facts the first time and 243 the second. The model does not give identical answers on every run, so improvements must be judged over several runs. Timings also vary: step 3 took 446 s in G1 and 687 s in G3 on the same document, and the context setting does not touch that step.

6.3 The Context-Size Experiment

Why long summaries were not shortened. In every run above the summary is only 8% to 10% shorter than the facts. For long RFPs, every point under the list headings was an exact copy of a fact. The reason is in the numbers of Section 5.9: the summary step saw about 2,050 of 17,496 tokens, so its answer was unusable for most headings, and the safety step copied every fact in. The high "100% backed by facts" score came partly from this copying.

We tested the fix on 11 golden documents with the same prompt and three settings of the context size (Table 6.3). "Within limits" counts documents whose sections stay inside the limits of the golden-summary standard.

Table 6.3: Effect of the context size on the summary (11 documents, average per document)

	Words per summary	Within limits	Backed by facts	Duplicate bullets
Default (4,096)	4,565	0 / 11	100.0%	25.3
`KLARO_NUM_CTX=16384`	3,054	4 / 11	96.6%	16.1
`KLARO_NUM_CTX=32768`	545	6 / 11	94.1%	0.9

The average summary shrinks from 4,565 to 545 words. The time for the summary step grows from 64 s (default) to 126 s (16,384) and 194 s (32,768), about three times longer. At 16,384, 5 documents still did not fit in the context. Grounding falls from 100% to 94.1%. We think this is because the model now writes its own sentences from the whole fact list instead of copying facts word for word, and the check compares words; we have not tested this. Only 6 of 11 summaries at 32,768 are within the limits, so the problem is reduced, not solved.

We also tried the setting on a German run (G3 against G1, same PDF). The summary shrank from 1,755 to 722 words and the summary step took 232 s instead of 54 s. But the share of facts that reached the summary fell from 98.3% to 64.8% (70 of 108 facts), for example the name and address of the tendering body and several data protection and termination clauses. The dates were kept. The grounding check flagged one point: the overview names the funding ministry as the buyer. So a shorter summary is not automatically a better one, and we have no human judgement of which of the two is better.

6.4 German Results

Translation. Of 581 segments in G1, 23 (4.0%) were flagged for a number that did not survive; of 589 segments in G2, 4 (0.7%). The flag rule is a rough check: times like 14:00 and two-digit years can trigger it wrongly. The G2 translation took 0.8 s, which looks like a cache hit (not checked), while G1 took 43.4 s. *Highlighting.* In G1, 90 of 104 points were linked to a passage with a score of 0.6 or more; in G2, 116

of 137; in G3, 26 of 40. The rest show no quote. *German view.* The English-to-German translation of G1's summary took 6.9 s for 110 segments (0.06 s per segment), and one bullet was flagged for a missing euro sign. *Issuer.* Both German runs got the issuer line wrong or only partly right: G1 names the funding ministry (the f-bb institute that issued the tender is named correctly in the overview), and G2 says "Federal Office" for the procurement office of the Federal Ministry of the Interior. We did not run the translation A/B test script in `scripts/translation_ab` (it needs both models), and no German speaker has judged the translation quality.

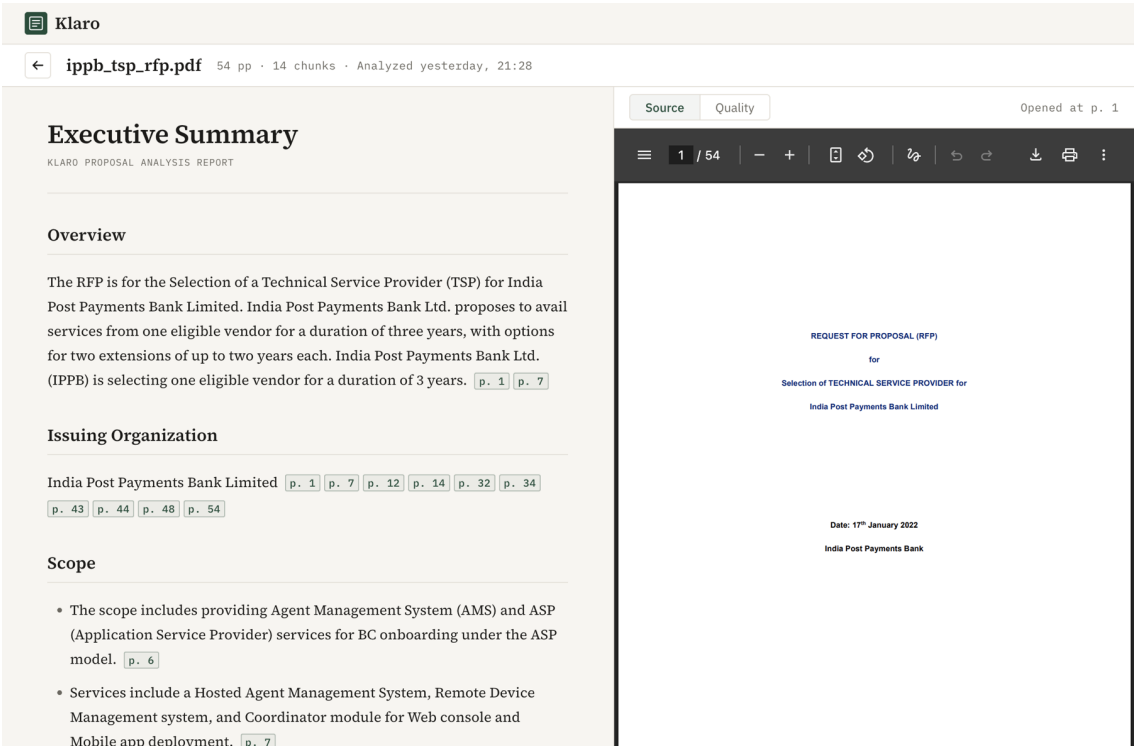


Figure 6.1: English RFP: the summary beside the original PDF. Clicking a page number opens that page (screenshot from an earlier run, before passage highlighting)

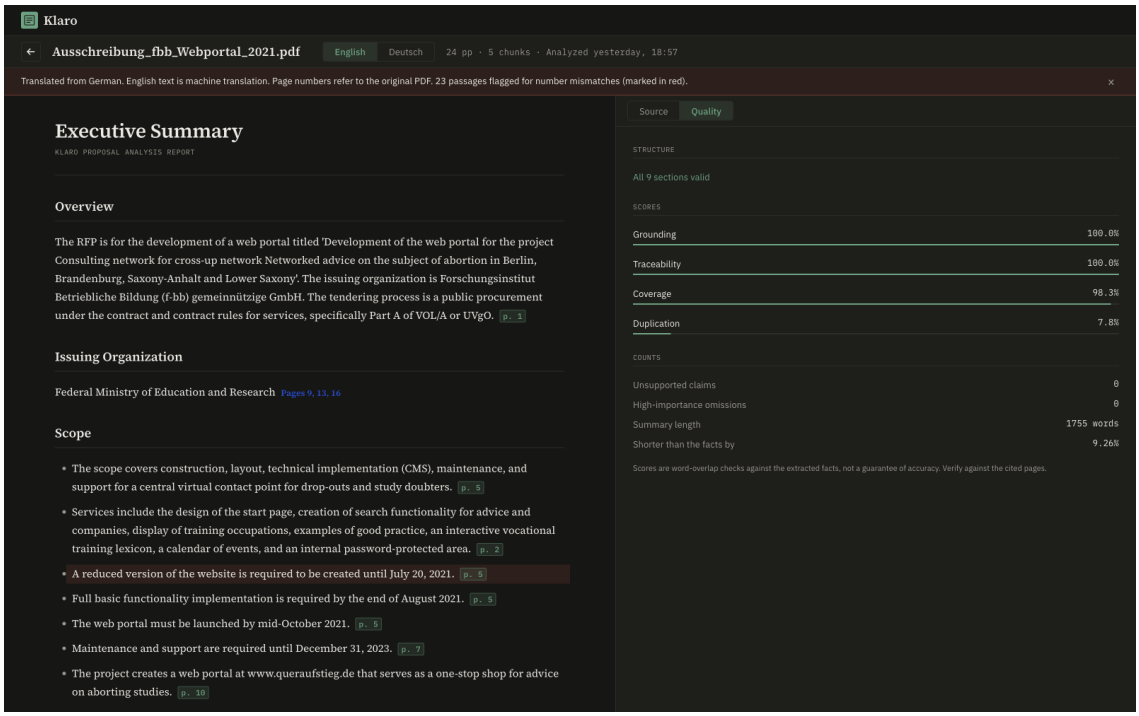


Figure 6.2: Results screen with the automatic quality checks and a flagged bullet

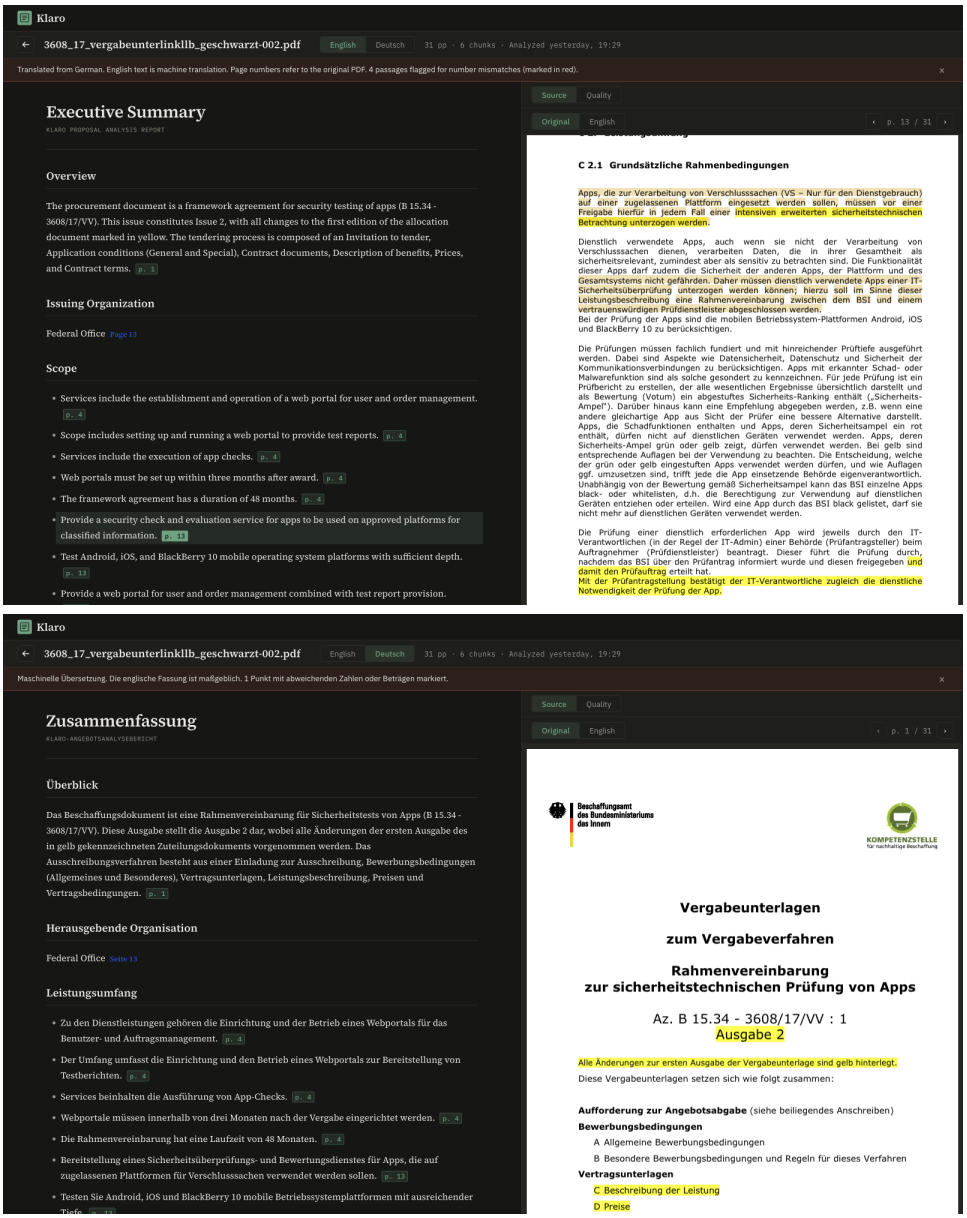


Figure 6.3: German RFP in the English view, with the "Translated from German" banner and a clicked point whose passage is highlighted in the PDF (top); the German view of another run (bottom)

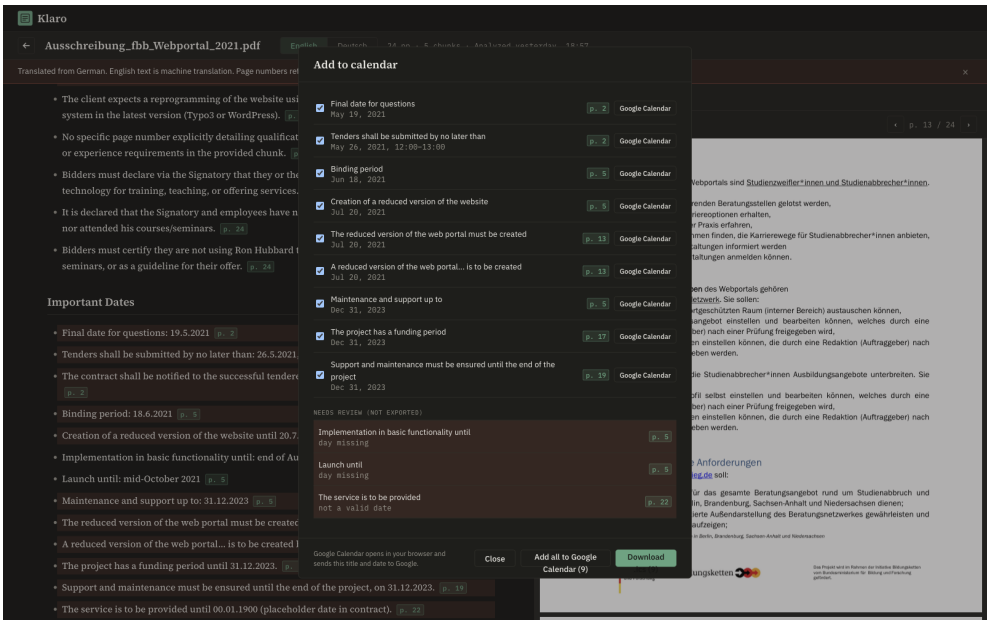


Figure 6.4: The “Add to calendar” preview, with items that need review

Chapter 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

Klaro works end to end on a laptop. An English RFP of 54 to 84 pages goes in, and 22 to 25 minutes later a nine-heading summary comes out, with a page number on every point. A German RFP of 24 to 31 pages takes 9 to 11 minutes, is translated on the laptop first, and each point can be traced to a highlighted passage in the PDF. Nothing is sent over the internet. We chose Qwen3.5 because it made the fewest wrong claims and is about a third of the size of Gemma. We also found why summaries of long RFPs were not shortened, and a larger context setting reduces the problem at the price of a longer run. The fine-tuning kit and the golden set are in place, but they have not yet improved the app.

7.2 Limitations

- The golden summaries are drafted by an AI and checked by machine. No human has verified them.
- The LoRA adapter has not been trained or tested on the real 4B model.
- The per-section adapter is not wired into the app, because the app writes the whole summary in one call.
- German translation quality has not been judged by a German speaker, and the translation A/B test has not been run.
- The issuer line is wrong or partial in both German runs, and some points cite a page where the passage could not be found.
- The context fix is off by default. With it only 6 of 11 summaries stay within the limits, the summary step is three to four times slower, and on one German run it kept only 65% of the facts. Its first run on that document timed out at the default 300 s limit.
- Only text-based PDFs work; tables lose their layout. Runs are slow (9 to 25 minutes).

7.3 Future Scope

- Full-length training on the university supercomputer next semester, with an evaluation on the 10 test documents.
- Human verification of the golden set, starting with the 10 test documents.
- A section-by-section summary step, so that the per-section adapter can be used in the app.
- Scanned PDFs with text recognition, and Word files.
- Two or three models for the user to choose from, so the user can pick the one that suits the laptop best. Later, more models can be packaged for download, and the user can use whichever fits the system specifications.
- A multilingual Klaro: more languages beyond English and German, with the same offline translation and page checking.
- A Proposal Assistant: a requirements checklist and a first proposal outline from the same reading of the RFP.
- Human in the loop interaction.

REFERENCES

[1] S. Beason, W. Hinton, Y. A. Salamah, and J. Salsman, "Automated analysis of RFPs using natural language processing (NLP) for the technology domain," *SMU Data Science Review*, vol. 5, no. 1, 2021, art. 1.

[2] A. J. C. Trappey *et al.*, "Intelligent RFQ summarization using natural language processing, text mining, and machine learning techniques," *Journal of Global Information Management*, vol. 30, no. 1, pp. 1–26, 2022.

[3] B. M. Knisely and H. H. Pavliscsak, "Research proposal content extraction using natural language processing and semi-supervised clustering: A demonstration and comparative analysis," *Scientometrics*, vol. 128, no. 5, pp. 3197–3224, 2023.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.

[5] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," arXiv:2004.05150, 2020.

[6] T. Douzon, S. Duffner, C. Garcia, and J. Espinas, "Long-range transformer architectures for document understanding," in *Proc. ICDAR 2023 Workshops on Document Analysis and Recognition*, ser. Lecture Notes in Computer Science. Springer, 2023, pp. 47–64.

[7] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[8] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On faithfulness and factuality in abstractive summarization," in *Proc. 58th Annu. Meeting Association for Computational Linguistics*, 2020, pp. 1906–1919.

[9] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[10] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474.

[11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[12] J. Tiedemann and S. Thottingal, "OPUS-MT: Building open translation services for the world," in *Proc. 22nd Annu. Conf. European Association for Machine Translation (EAMT)*, 2020, pp. 479–480.

[13] "Ollama: Get up and running with large language models locally," https://github.com/ollama/ollama.

[14] "qwen3.5 model library entry," https://ollama.com/library/qwen3.5.

[15] "pypdf documentation," https://pypdf.readthedocs.io.

[16] "Pydantic documentation," https://docs.pydantic.dev.

[17] "FastAPI documentation," https://fastapi.tiangolo.com.

[18] "Electron documentation: Security," https://www.electronjs.org/docs/latest/tutorial/security.

[19] "LangGraph documentation," https://langchain-ai.github.io/langgraph/.